main

September 28, 2026

[1]:
```
import pandas as pd
```

[2]:
```
df = pd.read_csv("sales_data.csv")
display(df.describe())
```

```
        Product_ID  Sales_Amount  Quantity_Sold    Unit_Cost   Unit_Price  \
count  1000.000000   1000.000000    1000.000000  1000.000000  1000.000000
mean   1050.128000   5019.265230      25.355000  2475.304550  2728.440120
std      29.573505   2846.790126      14.159006  1417.872546  1419.399839
min    1001.000000    100.120000       1.000000    60.280000   167.120000
25%    1024.000000   2550.297500      13.000000  1238.380000  1509.085000
50%    1051.000000   5019.300000      25.000000  2467.235000  2696.400000
75%    1075.000000   7507.445000      38.000000  3702.865000  3957.970000
max    1100.000000   9989.040000      49.000000  4995.300000  5442.150000

          Discount
count  1000.00000
mean      0.15239
std       0.08720
min       0.00000
25%       0.08000
50%       0.15000
75%       0.23000
max       0.30000
```

[3]:
```
display(df.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Product_ID            1000 non-null   int64
 1   Sale_Date             1000 non-null   str
 2   Sales_Rep             1000 non-null   str
 3   Region                1000 non-null   str
 4   Sales_Amount          1000 non-null   float64
 5   Quantity_Sold         1000 non-null   int64
 6   Product_Category      1000 non-null   str
```

```
 7   Unit_Cost             1000 non-null   float64
 8   Unit_Price            1000 non-null   float64
 9   Customer_Type         1000 non-null   str
 10  Discount              1000 non-null   float64
 11  Payment_Method        1000 non-null   str
 12  Sales_Channel         1000 non-null   str
 13  Region_and_Sales_Rep  1000 non-null   str
dtypes: float64(4), int64(2), str(8)
memory usage: 109.5 KB
```

None

[4]:
```
print(df.shape)
```

(1000, 14)

[5]:
```
display(df.head())
print("\n\n")
display(df.tail())
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	\
0	1052	2023-02-03	Bob	North	5053.97	18	
1	1093	2023-04-21	Bob	West	4384.02	17	
2	1015	2023-09-21	David	South	4631.23	30	
3	1072	2023-08-24	Bob	South	2167.94	39	
4	1061	2023-03-24	Charlie	East	3750.20	13	

	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	\
0	Furniture	152.75	267.22	Returning	0.09	
1	Furniture	3816.39	4209.44	Returning	0.11	
2	Food	261.56	371.40	Returning	0.20	
3	Clothing	4330.03	4467.75	New	0.02	
4	Electronics	637.37	692.71	New	0.08	

	Payment_Method	Sales_Channel	Region_and_Sales_Rep
0	Cash	Online	North-Bob
1	Cash	Retail	West-Bob
2	Bank Transfer	Retail	South-David
3	Credit Card	Retail	South-Bob
4	Credit Card	Online	East-Charlie

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	\
995	1010	2023-04-15	Charlie	North	4733.88	4	
996	1067	2023-09-07	Bob	North	4716.36	37	
997	1018	2023-04-27	David	South	7629.70	17	
998	1100	2023-12-20	David	West	1629.47	39	

```
999        1086  2023-08-16     Alice   East       4923.93             48

    Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount  \
995             Food    4943.03     5442.15     Returning      0.29
996         Clothing    1754.32     1856.40           New      0.21
997         Clothing     355.72      438.27     Returning      0.06
998      Electronics    3685.03     3743.39           New      0.01
999             Food    2632.58     2926.68     Returning      0.14

    Payment_Method Sales_Channel Region_and_Sales_Rep
995           Cash        Online        North-Charlie
996  Bank Transfer        Retail            North-Bob
997  Bank Transfer        Online          South-David
998  Bank Transfer        Online           West-David
999           Cash        Online           East-Alice
```

[6]:
```
print(df.nunique())
```

```
Product_ID              100
Sale_Date               340
Sales_Rep                 5
Region                    4
Sales_Amount            998
Quantity_Sold            49
Product_Category          4
Unit_Cost               998
Unit_Price             1000
Customer_Type             2
Discount                 31
Payment_Method            3
Sales_Channel             2
Region_and_Sales_Rep     20
dtype: int64
```

[7]:
```
non_numeric_df = df.select_dtypes(exclude=["number"])
for cols in non_numeric_df.columns:
    display(df[cols].unique())
```

```
<StringArray>
['2023-02-03', '2023-04-21', '2023-09-21', '2023-08-24', '2023-03-24',
 '2023-02-11', '2023-04-11', '2023-01-06', '2023-06-29', '2023-10-09',
 …
 '2023-06-23', '2023-02-19', '2023-01-24', '2023-07-24', '2023-04-26',
 '2023-11-13', '2023-12-14', '2023-08-26', '2023-08-09', '2023-07-26']
Length: 340, dtype: str
```

```
<StringArray>
['Bob', 'David', 'Charlie', 'Eve', 'Alice']
Length: 5, dtype: str
```

```
<StringArray>
['North', 'West', 'South', 'East']
Length: 4, dtype: str

<StringArray>
['Furniture', 'Food', 'Clothing', 'Electronics']
Length: 4, dtype: str

<StringArray>
['Returning', 'New']
Length: 2, dtype: str

<StringArray>
['Cash', 'Bank Transfer', 'Credit Card']
Length: 3, dtype: str

<StringArray>
['Online', 'Retail']
Length: 2, dtype: str

<StringArray>
[    'North-Bob',      'West-Bob',   'South-David',     'South-Bob',
  'East-Charlie',  'West-Charlie',     'South-Eve',     'North-Eve',
      'West-Eve',   'South-Alice', 'South-Charlie',   'North-David',
      'East-Bob',    'West-David',      'East-Eve',   'North-Alice',
    'East-David',    'West-Alice',    'East-Alice', 'North-Charlie']
Length: 20, dtype: str
```

[8]:
```
print(df.duplicated().sum())
```

```
0
```

[9]:
```
numeric_df = df.select_dtypes(include=["number"])
for cols in numeric_df.columns:
    print(df[cols].duplicated().sum())
```

```
900
2
951
2
0
969
```

[10]:
```
df['Sale_Date'] =  pd.to_datetime(df['Sale_Date'])
print("Start:", df["Sale_Date"].min())
print("End:", df["Sale_Date"].max())
```

```
Start: 2023-01-01 00:00:00
End: 2024-01-01 00:00:00
```

[11]:
```
print(df['Sale_Date'].dt.month.value_counts().sort_index())
```

```
Sale_Date
1     103
2      75
3      80
4      81
5      72
6      92
7      68
8      93
9      68
10     88
11     95
12     85
Name: count, dtype: int64
```

[12]:
```
operations = ["mean","median","min","max"]
for ops in operations:
    method = getattr(df['Sales_Amount'],ops)
    print(f"{ops}: {method()}")
```

```
mean: 5019.265229999999
median: 5019.3
min: 100.12
max: 9989.04
```

[13]:
```
for region in df['Region'].unique():
    print(region)
```

```
North
West
South
East
```

[14]:
```
region_sales = (df.groupby("Region")["Sales_Amount"].sum().
 ↪sort_values(ascending=False))
display(region_sales)
```

```
Region
North    1369612.51
East     1259792.93
West     1235608.93
South    1154250.86
Name: Sales_Amount, dtype: float64
```

[15]:
```
rep_sales = (df.groupby("Sales_Rep")["Sales_Amount"].
 ↪agg(["sum","mean","count"]).sort_values("sum",ascending=False))
rep_sales_mean = (df.groupby("Sales_Rep")["Sales_Amount"].
 ↪agg(["sum","mean","median","count"]).sort_values("mean",ascending=False))
print(rep_sales)
```

```
print("\n\n")
print(rep_sales_mean)
```

```
                  sum         mean  count
Sales_Rep
David      1141737.36  5142.961081    222
Bob        1080990.63  5197.070337    208
Eve         970183.99  4642.028660    209
Alice       965541.77  5028.863385    192
Charlie     860811.48  5093.559053    169
```

```
                  sum         mean    median  count
Sales_Rep
Bob        1080990.63  5197.070337  5399.105    208
David      1141737.36  5142.961081  5510.905    222
Charlie     860811.48  5093.559053  4947.280    169
Alice       965541.77  5028.863385  4974.785    192
Eve         970183.99  4642.028660  4333.250    209
```

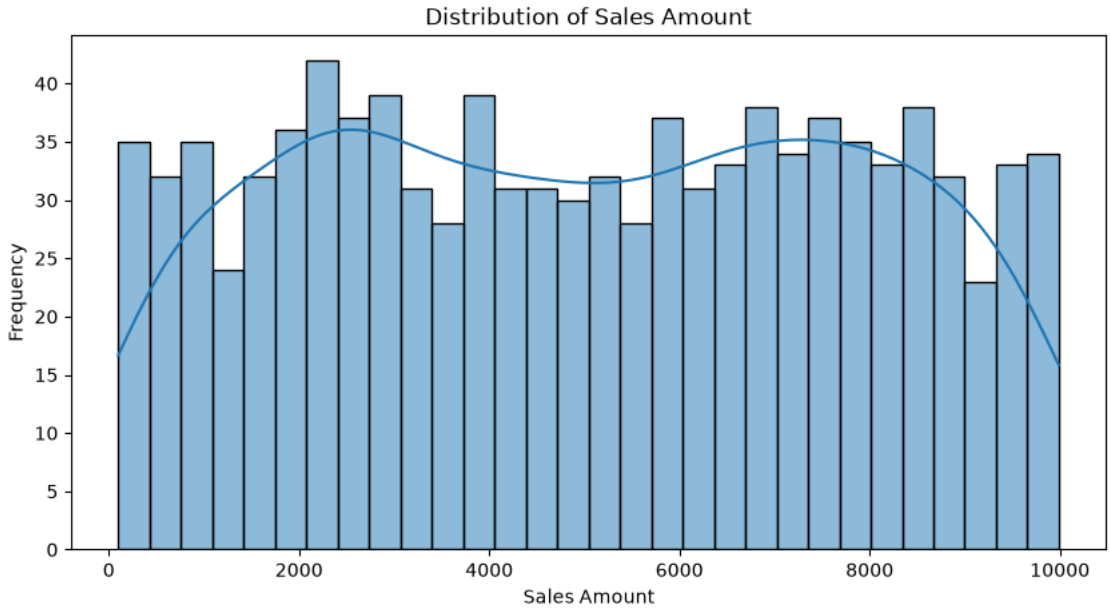
[16]:
```
category_region = pd.pivot_table(
    df,
    values = "Sales_Amount",
    index = "Region",
    columns = "Product_Category",
    aggfunc="sum"
)
display(category_region)
```

```
Product_Category   Clothing  Electronics       Food  Furniture
Region
East               356670.40    303101.42  325864.87  274156.24
North              372977.22    342666.29  258985.65  394983.35
South              269517.74    293663.96  301187.51  289881.65
West               314309.00    304067.97  315735.51  301496.45
```

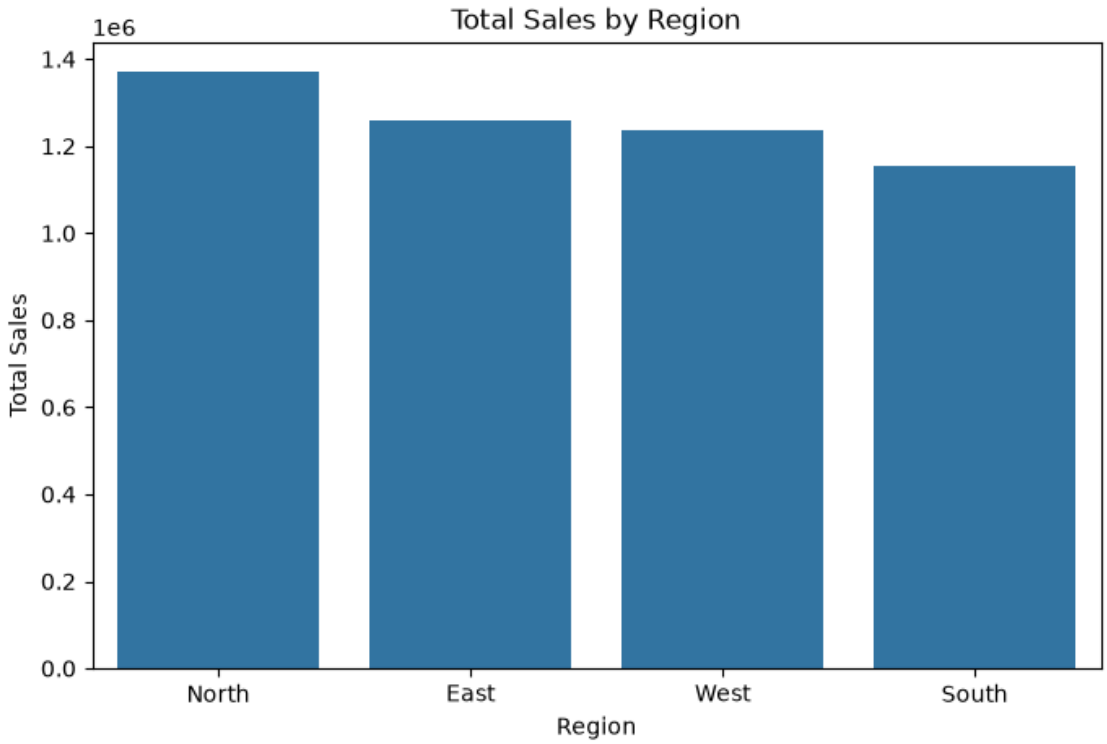
[17]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

[18]:
```
plt.figure(figsize=(10,5))
sns.histplot(
    data=df,
    x="Sales_Amount",
    bins=30,
    kde=True
)
plt.title("Distribution of Sales Amount")
```

```
plt.xlabel("Sales Amount")
plt.ylabel("Frequency")
plt.show()
```



[19]:
```
plt.figure(figsize=(8,5))
sns.barplot(
    x=region_sales.index,
    y=region_sales.values
)
plt.title("Total Sales by Region")
plt.xlabel("Region")
plt.ylabel("Total Sales")
plt.show()
```

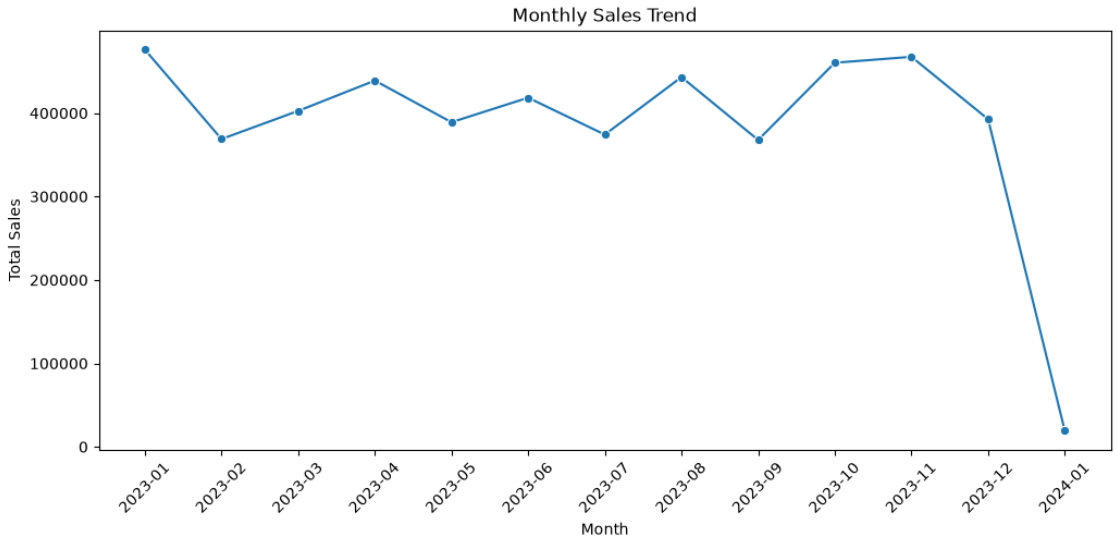


[20]:
```
monthly_sales = (
    df.groupby(df['Sale_Date'].dt.to_period("M"))["Sales_Amount"].sum()
)
display(monthly_sales)
```

```
Sale_Date
2023-01    476092.36
2023-02    368919.36
2023-03    402638.77
2023-04    438992.61
2023-05    389078.76
2023-06    418458.34
2023-07    374242.88
2023-08    443171.28
2023-09    367837.60
2023-10    460378.78
2023-11    467482.90
2023-12    392643.58
2024-01     19328.01
Freq: M, Name: Sales_Amount, dtype: float64
```

[21]:
```
plt.figure(figsize=(12,5))
sns.lineplot(
```

```
    x=monthly_sales.index.astype(str),
    y=monthly_sales.values,
    marker="o"
)
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.show()
```



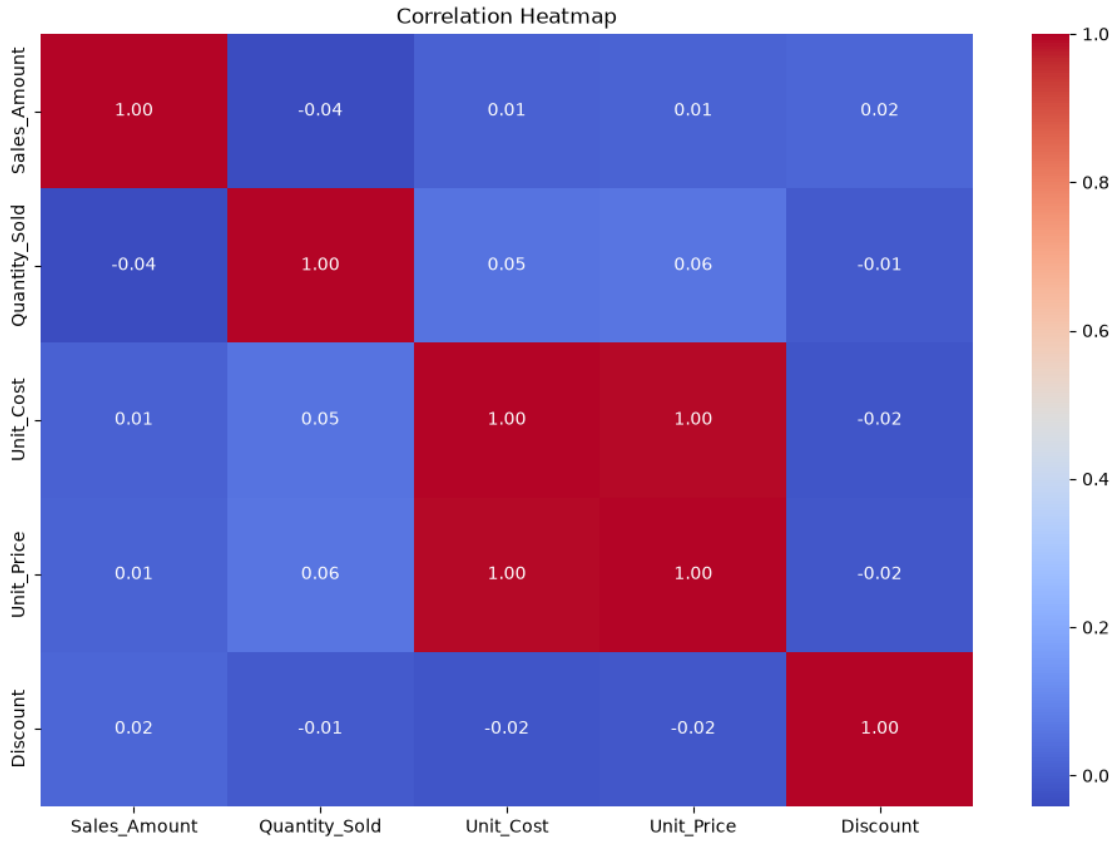
[22]:
```
plt.figure(figsize=(8,5))
sns.scatterplot(
    data=df,
    x="Quantity_Sold",
    y="Sales_Amount"
)
plt.title("Quantity Sold vs Sales Amount")
plt.xlabel("Quantity Sold")
plt.ylabel("Sales Amount")
plt.show()
```



[23]:
```
numeric_cols =␣
  ↪["Sales_Amount","Quantity_Sold","Unit_Cost","Unit_Price","Discount"]
corr = df[numeric_cols].corr()
display(corr)
```

	Sales_Amount	Quantity_Sold	Unit_Cost	Unit_Price	Discount
Sales_Amount	1.000000	-0.041599	0.009770	0.011812	0.023153
Quantity_Sold	-0.041599	1.000000	0.053888	0.057296	-0.007806
Unit_Cost	0.009770	0.053888	1.000000	0.995056	-0.017741
Unit_Price	0.011812	0.057296	0.995056	1.000000	-0.017027
Discount	0.023153	-0.007806	-0.017741	-0.017027	1.000000

[24]:
```
plt.figure(figsize=(12,8))
sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
plt.title("Correlation Heatmap")
plt.show()
```



[25]:
```
print(
    df[df["Sale_Date"] >= "2024-01-01"].shape
)
display(df[df["Sale_Date"] >= "2024-01-01"])
```

```
(3, 14)
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	\
317	1018	2024-01-01	Alice	West	8579.72	24	
627	1078	2024-01-01	David	East	8377.57	42	
873	1098	2024-01-01	Bob	South	2370.72	11	

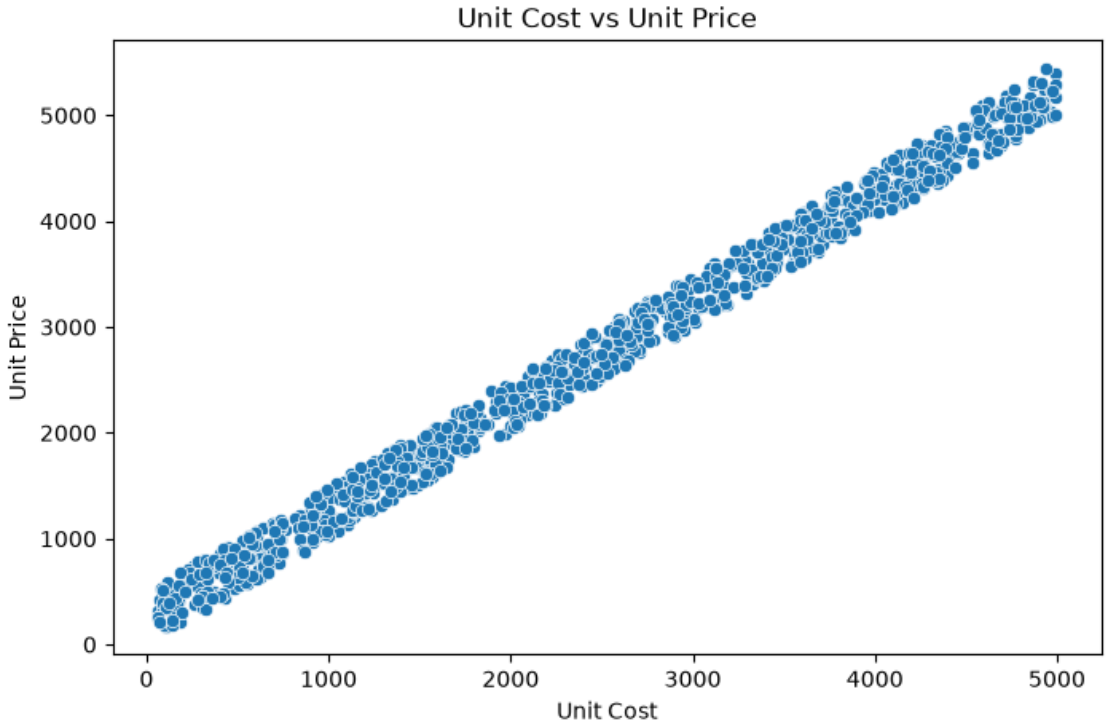
	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	\
317	Electronics	1362.78	1729.81	New	0.29	
627	Food	63.41	314.93	Returning	0.03	
873	Clothing	213.41	503.36	Returning	0.27	

	Payment_Method	Sales_Channel	Region_and_Sales_Rep
317	Bank Transfer	Online	West-Alice
627	Cash	Retail	East-David
873	Bank Transfer	Online	South-Bob

[26]:
```
monthly_analysis = df.groupby(
    df["Sale_Date"].dt.to_period("M")
).agg(
    transactions=("Sales_Amount","count"),
    revenue=("Sales_Amount","sum"),
    avg_sales=("Sales_Amount","mean")
)
display(monthly_analysis)
```

Sale_Date	transactions	revenue	avg_sales
2023-01	100	476092.36	4760.923600
2023-02	75	368919.36	4918.924800
2023-03	80	402638.77	5032.984625
2023-04	81	438992.61	5419.661852
2023-05	72	389078.76	5403.871667
2023-06	92	418458.34	4548.460217
2023-07	68	374242.88	5503.571765
2023-08	93	443171.28	4765.282581
2023-09	68	367837.60	5409.376471
2023-10	88	460378.78	5231.577045
2023-11	95	467482.90	4920.872632
2023-12	85	392643.58	4619.336235
2024-01	3	19328.01	6442.670000

[27]:
```
plt.figure(figsize=(8,5))
sns.scatterplot(
    data=df,
    x="Unit_Cost",
    y="Unit_Price"
)
plt.title("Unit Cost vs Unit Price")
plt.xlabel("Unit Cost")
plt.ylabel("Unit Price")
plt.show()
```



[28]:
```
df["Gross_Profit"] = (
    df["Unit_Price"] - df["Unit_Cost"]
) * df["Quantity_Sold"]
```

[29]:
```
df["Revenue_Before_Discount"] = (
    df["Unit_Price"] * df["Quantity_Sold"]
)
```

[30]:
```
display(
    df[["Sales_Amount","Revenue_Before_Discount","Gross_Profit","Discount"]].
 ↪describe()
)
```

	Sales_Amount	Revenue_Before_Discount	Gross_Profit	Discount
count	1000.000000	1000.000000	1000.000000	1000.00000
mean	5019.265230	70329.940710	6487.847070	0.15239
std	2846.790126	57783.299413	5444.188837	0.08720
min	100.120000	211.180000	17.360000	0.00000
25%	2550.297500	23363.535000	1892.302500	0.08000
50%	5019.300000	54519.175000	5236.825000	0.15000
75%	7507.445000	105855.845000	9935.082500	0.23000
max	9989.040000	252147.360000	23441.110000	0.30000

[31]:
```
df["Calculated_Sales"] = (
    df["Unit_Price"] * df["Quantity_Sold"] * (1-df["Discount"])
)
```

[32]:
```
df["Difference"] = (
    df["Sales_Amount"] - df["Calculated_Sales"]
)
display(
    df["Difference"].describe()
)
print(
    (df["Difference"].abs() < 0.01).sum()
)
```

```
count      1000.000000
mean     -54666.908054
std       49866.811468
min     -239923.616400
25%      -85767.534350
50%      -40647.065950
75%      -15057.454825
max        8845.421800
Name: Difference, dtype: float64

0
```

[33]:
```
display(df["Unit_Cost"])
print("\n\n")
display(df["Unit_Price"])
```

```
0       152.75
1      3816.39
2       261.56
3      4330.03
4       637.37
        ...
995    4943.03
996    1754.32
997     355.72
998    3685.03
999    2632.58
Name: Unit_Cost, Length: 1000, dtype: float64



0       267.22
1      4209.44
2       371.40
```

```
3      4467.75
4       692.71
        …
995    5442.15
996    1856.40
997     438.27
998    3743.39
999    2926.68
Name: Unit_Price, Length: 1000, dtype: float64
```

[34]:
```
numeric_cols =␣
  ↪["Sales_Amount","Quantity_Sold","Unit_Cost","Unit_Price","Discount"]
corr = df[numeric_cols].corr()["Sales_Amount"].sort_values()
display(corr)
```

```
Quantity_Sold   -0.041599
Unit_Cost        0.009770
Unit_Price       0.011812
Discount         0.023153
Sales_Amount     1.000000
Name: Sales_Amount, dtype: float64
```

[35]:
```
product_analysis = df.groupby("Product_ID").agg(
    transactions=("Product_ID","count"),
    avg_sales=("Sales_Amount","mean"),
    avg_quantity=("Quantity_Sold","mean"),
    avg_cost=("Unit_Cost","mean"),
    avg_price=("Unit_Price","mean"),
    avg_discount=("Discount","mean")
)
display(product_analysis.head(20))
print("\n\n")
display(product_analysis.describe())
```

Product_ID	transactions	avg_sales	avg_quantity	avg_cost	avg_price	\
1001	16	5063.583750	23.375000	2371.550625	2670.316875	
1002	13	5526.254615	23.692308	2391.833846	2614.109231	
1003	12	5038.583333	21.250000	2570.984167	2778.828333	
1004	11	6431.328182	20.909091	3145.320909	3393.677273	
1005	11	4373.972727	20.090909	2244.798182	2502.542727	
1006	8	4669.636250	26.250000	2923.167500	3210.396250	
1007	8	3052.470000	17.875000	2838.547500	3130.411250	
1008	13	5121.055385	31.307692	2313.291538	2599.936154	
1009	10	4383.435000	23.800000	2326.819000	2651.391000	
1010	6	5038.155000	31.166667	3383.563333	3609.551667	
1011	4	4774.857500	18.750000	3032.512500	3285.690000	
1012	13	4766.537692	20.230769	2410.393077	2685.359231	
1013	10	5876.751000	29.400000	2250.030000	2450.922000	

```
1014                 10  5963.315000     24.500000  1717.750000  1964.559000
1015                 10  4242.430000     22.400000  2527.437000  2686.232000
1016                 13  4919.726154     18.846154  2748.103846  3003.164615
1017                 15  3723.636667     21.066667  1919.154000  2209.248667
1018                  9  4878.463333     17.777778  1906.383333  2072.588889
1019                 14  4988.985000     25.642857  2611.036429  2893.233571
1020                  7  7086.165714     28.714286  3056.972857  3269.044286

            avg_discount
Product_ID
1001            0.129375
1002            0.156154
1003            0.174167
1004            0.152727
1005            0.095455
1006            0.195000
1007            0.208750
1008            0.170000
1009            0.139000
1010            0.125000
1011            0.147500
1012            0.143846
1013            0.120000
1014            0.152000
1015            0.156000
1016            0.195385
1017            0.154667
1018            0.141111
1019            0.148571
1020            0.157143



       transactions    avg_sales  avg_quantity     avg_cost    avg_price  \
count    100.000000   100.000000    100.000000   100.000000   100.000000
mean      10.000000  4974.350202     25.397600  2469.820470  2721.419516
std        3.393398   987.570348      4.879227   435.463561   444.049195
min        3.000000  2014.762500     13.000000  1589.885000  1814.071111
25%        8.000000  4442.941458     22.433333  2235.164006  2479.841000
50%       10.000000  4992.991071     25.821429  2436.990268  2682.168615
75%       12.000000  5661.210417     28.881579  2710.623421  3005.236462
max       20.000000  7408.801818     39.714286  3590.179167  3848.759286

       avg_discount
count    100.000000
mean       0.150857
std        0.027893
```

```
min         0.073333
25%         0.135375
50%         0.150992
75%         0.167649
max         0.220000
```

[38]:
```
product_consistency = df.groupby("Product_ID").agg(
    price_min=("Unit_Price","min"),
    price_max=("Unit_Price","max"),
    price_avg=("Unit_Price","mean"),
    price_std=("Unit_Price","std"),

    cost_min=("Unit_Cost","min"),
    cost_max=("Unit_Cost","max"),
    cost_avg=("Unit_Cost","mean"),
    cost_std=("Unit_Cost","std"),

    quantity_min=("Quantity_Sold","min"),
    quantity_max=("Quantity_Sold","max"),
    quantity_avg=("Quantity_Sold","mean"),
    quantity_std=("Quantity_Sold","std")
)
display(product_consistency.head(20))
```

```
            price_min  price_max    price_avg    price_std  cost_min  \
Product_ID
1001           682.24    5316.13  2670.316875  1313.216839    442.11
1002           557.69    4784.28  2614.109231  1583.099264    117.63
1003           457.84    5227.93  2778.828333  1600.153028    310.96
1004           211.18    5088.06  3393.677273  1644.008151    182.99
1005           385.83    5026.19  2502.542727  1796.814471    122.50
1006          1098.97    4925.17  3210.396250  1444.283765    834.32
1007          1608.76    4633.10  3130.411250  1217.968739   1122.07
1008           429.29    5075.44  2599.936154  1476.791794     68.99
1009           733.00    4710.10  2651.391000  1378.140257    359.41
1010          1306.02    5442.15  3609.551667  1523.244828   1150.30
1011          1510.30    5107.43  3285.690000  1851.154311   1324.52
1012           803.71    4528.77  2685.359231  1291.346048    325.75
1013          1258.30    4642.83  2450.922000  1310.423623    919.58
1014           316.39    4747.07  1964.559000  1438.480398    132.47
1015           371.40    4758.11  2686.232000  1538.893707    261.56
1016           338.94    4954.39  3003.164615  1656.858368     88.44
1017           167.12    5287.05  2209.248667  1579.914635    102.23
1018           438.27    5011.43  2072.588889  1399.222595    355.72
1019           487.03    5145.88  2893.233571  1501.443296     97.24
1020           675.91    4805.17  3269.044286  1642.671822    184.05

            cost_max     cost_avg     cost_std  quantity_min  quantity_max  \
```

Product_ID					
1001	4865.33	2371.550625	1248.875666	2	47
1002	4646.55	2391.833846	1588.460792	9	44
1003	4974.14	2570.984167	1615.704163	3	45
1004	4881.63	3145.320909	1563.458351	1	39
1005	4737.06	2244.798182	1808.511416	2	46
1006	4562.58	2923.167500	1453.233190	11	49
1007	4310.31	2838.547500	1230.034935	1	43
1008	4739.13	2313.291538	1511.624607	22	49
1009	4364.35	2326.819000	1435.625360	1	47
1010	4943.03	3383.563333	1446.358033	4	49
1011	4906.17	3032.512500	1913.425460	2	38
1012	4190.83	2410.393077	1300.526582	1	47
1013	4625.33	2250.030000	1278.711102	16	46
1014	4705.46	1717.750000	1484.270891	3	49
1015	4680.35	2527.437000	1522.869538	1	35
1016	4906.85	2748.103846	1669.306000	2	46
1017	4867.64	1919.154000	1543.418822	1	43
1018	4660.82	1906.383333	1371.384831	1	28
1019	4977.78	2611.036429	1516.109247	12	45
1020	4772.03	3056.972857	1707.588827	4	45

Product_ID	quantity_avg	quantity_std
1001	23.375000	14.827340
1002	23.692308	9.612358
1003	21.250000	12.828413
1004	20.909091	14.494513
1005	20.090909	14.625010
1006	26.250000	13.925824
1007	17.875000	13.736682
1008	31.307692	8.239990
1009	23.800000	15.068731
1010	31.166667	17.127950
1011	18.750000	15.217862
1012	20.230769	13.797064
1013	29.400000	11.423173
1014	24.500000	18.863545
1015	22.400000	10.627010
1016	18.846154	15.296220
1017	21.066667	13.619664
1018	17.777778	9.066115
1019	25.642857	10.710219
1020	28.714286	14.499589

[41]:
```
price_range = df.groupby("Product_ID")["Unit_Price"].
  ↪agg(["min","max","mean","std"])
```

```
display(price_range.sort_values("std",ascending=False).head(20))
```

Product_ID	min	max	mean	std
1043	759.32	5079.68	2704.194000	2060.608538
1031	818.14	4382.32	2970.643333	1894.059894
1011	1510.30	5107.43	3285.690000	1851.154311
1005	385.83	5026.19	2502.542727	1796.814471
1045	613.09	5129.19	2593.176667	1783.492209
1087	297.69	4261.11	2603.950000	1770.520606
1064	488.23	5118.83	2706.222222	1755.591686
1079	675.54	5080.74	2409.051667	1706.671069
1023	336.37	4825.92	2526.041538	1705.525381
1072	693.40	4645.17	2404.840000	1694.715070
1021	584.44	5309.32	2602.622000	1691.201743
1070	451.81	4166.95	2494.622857	1678.422444
1077	258.92	4724.70	2269.877143	1676.436494
1016	338.94	4954.39	3003.164615	1656.858368
1084	381.92	4645.64	2294.748750	1649.711176
1094	700.25	4695.54	3021.210000	1648.975036
1004	211.18	5088.06	3393.677273	1644.008151
1100	528.49	5402.28	2341.298889	1643.635335
1020	675.91	4805.17	3269.044286	1642.671822
1065	498.31	4612.49	2602.065000	1627.002973

[ ]: